Table of Contents

Quick Start

The example code provided in this quick start guide is for educational and demonstration purposes only. It may not represent best practices for production use. This quick start was last updated for **UDownloadManager v1.0.0-preview.1**.

Breaking Changes Notice

If you've just updated the package, it is recommended to check the *changelogs* for information on breaking changes.

Sample

This sample demonstrates how an app can download Gemma 4 E2B at runtime and used with UAI.LiteRTLM.

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Threading.Tasks;
using UnityEngine;
using Uralstech.UAI.LiteRTLM;
using Uralstech.UDownloadManager;

#nullable enable

/// <summary>
/// Demonstrates how to:
/// 1. Queue a download using UDownloadManager.
/// 2. Resume the download after an application restart.
/// 3. Wait for the download to complete.
/// 4. Use the downloaded file.
///
/// The LiteRTLM code is only an example consumer of the downloaded model.
/// Replace RunModelAsync() with your own logic if you're downloading a
/// different type of file.
/// </summary>
public sealed class DownloadModelExample : MonoBehaviour
{
    [Header("Download")]

    [SerializeField]
    private string _modelUrl =
        "https://huggingface.co/litert-community/gemma-4-E2B-it-litert-
lm/resolve/main/gemma-4-E2B-it.litertlm?download=true";
```

```
    [SerializeField]
    private string _modelPath = "models/model.litertlm";

    [SerializeField]
    private string _downloadIdKey = "ModelDownloadId";

    private string ModelPath => Path.Combine(Application.persistentDataPath,
_modelPath);

    private async void Start()
    {
        try
        {
            // The model is already available.
            if (File.Exists(ModelPath))
            {
                Debug.Log("Model already exists.");
                await RunModelAsync(ModelPath);
                return;
            }

            DownloadId downloadId = GetExistingDownload() ?? StartDownload();

            await WaitForDownloadAsync(downloadId);

            if (File.Exists(ModelPath))
                await RunModelAsync(ModelPath);
        }
        catch (Exception ex)
        {
            Debug.LogException(ex);
        }
    }

    /// <summary>
    /// Attempts to recover a previously queued download.
    /// Returns null if a new download should be started.
    /// </summary>
    private DownloadId? GetExistingDownload()
    {
        string? savedId = PlayerPrefs.GetString(_downloadIdKey, null);

        if (string.IsNullOrEmpty(savedId) || !long.TryParse(savedId, out
long androidId))
            return null;
```

```
        DownloadId downloadId = DownloadId.FromAndroidId(androidId);

        DownloadFilter filter = new();
        filter.ByIds(downloadId);

        IReadOnlyList<Download> downloads =
DownloadManager.Instance.QueryDownloads(filter);

        if (downloads.Count == 0)
            return null;

        AndroidDownload download = (AndroidDownload)downloads[0];

        switch (download.Status)
        {
            case DownloadStatus.Successful:
                Debug.Log("Download already completed.");
                return downloadId;

            case DownloadStatus.Failed:
                Debug.LogWarning($"Previous download
failed: {download.FailReason}");
                return null;

            default:
                Debug.Log("Resuming existing download.");
                return downloadId;
        }
    }

    /// <summary>
    /// Queues a new download and stores its ID so it can be resumed later.
    /// </summary>
    private DownloadId StartDownload()
    {
        AndroidDownloadRequest request = new(_modelUrl);

        request.SetAllowedOverMetered(true);
        request.SetAllowedOverRoaming(true);

        request.SetTitle("Downloading model");
        request.SetDescription("Downloading LiteRT-LM model");

        request.SetDestination(ModelPath);
```

```
        if (!DownloadManager.Instance.TryEnqueueDownload(request, out
DownloadId? downloadId))
            throw new InvalidOperationException("Failed to enqueue download.");

        PlayerPrefs.SetString(_downloadIdKey, downloadId.ToAndroidId().ToString());
        PlayerPrefs.Save();

        Debug.Log($"Download queued ({downloadId.ToAndroidId()}).");

        return downloadId;
    }

    /// <summary>
    /// Polls the DownloadManager until the download succeeds or fails.
    /// </summary>
    private async Task WaitForDownloadAsync(DownloadId downloadId)
    {
        DownloadFilter filter = new();
        filter.ByIds(downloadId);

        while (true)
        {
            await Awaitable.WaitForSecondsAsync(2f);

            IReadOnlyList<Download> downloads =
DownloadManager.Instance.QueryDownloads(filter);

            if (downloads.Count == 0)
                throw new Exception("Download no longer exists.");

            AndroidDownload download = (AndroidDownload)downloads[0];

            Debug.Log(
                $"Status: {download.Status} | " +
                $"Downloaded: {download.DownloadedSoFar} bytes");

            switch (download.Status)
            {
                case DownloadStatus.Successful:
                    Debug.Log("Download completed.");
                    return;

                case DownloadStatus.Failed:
                    throw new Exception($"Download failed: {download.FailReason}");
            }
        }
```

```
    }

    #region Example consumer

    /// <summary>
    /// Example showing how the downloaded model can be used.
    /// Replace this with your own logic if you're downloading another file.
    /// </summary>
    private static async Task RunModelAsync(string modelPath)
    {
        LiteRTLMNativeLogging.SetMinLogLevel(LogSeverity.Info);

        string cacheDir = Path.Combine(Application.temporaryCachePath,
"modelCache");
        Directory.CreateDirectory(cacheDir);

        using EngineSettings settings = new(modelPath, BackendNames.GPU);
        settings.SetEnableSpeculativeDecoding(true);
        settings.SetCacheDir(cacheDir);

        using Engine engine = new(settings);

        using ThinkingConfig thinking = new();
        thinking.SetEnableThinking(false);

        using ConversationConfig config = new();
        config.SetThinkingConfig(thinking);

        using Conversation conversation = new(engine, config);

        Debug.Log("Engine initialized.");

        const string message =
            "{\"role\":\"user\",\"content\":\"What is the tallest building in the
world?\"}";

        TaskCompletionSource<bool> completion = new();

        void OnChunk(StreamChunk chunk)
        {
            Debug.Log(chunk.GetText());

            if (chunk.IsFinal())
                completion.TrySetResult(true);
        }
```

```
        if (conversation.SendMessageStream(OnChunk, message) == 0)
            await completion.Task;
    }

    #endregion
}
```